Sensor SDE Coding Project

Threaded File Finder Candidate Instructions

Problem Statement

You are to write a command line application that searches a directory tree (i.e. - including subdirectories) for filenames that contain anywhere in them a specified substring.

The application takes the directory to search as the first command line argument, followed by one or more substrings (literals, not wildcards) to search for.

```
file-finder <dir> <substring1>[<substring2> [<substring3>]...]
```

For example:

```
file-finder tmp aaa bbb ccc
```

This would search the directory tree rooted at “tmp” for filenames (only the filename, not the rest of the path) that contain “aaa,” “bbb” or “ccc” anywhere in them.

Each filename that matches a substring gets added to a container of some type to record them.

The program should dump the container’s contents periodically (frequency at your discretion) to the console, clear the contents of the container, then resume scanning.

Additionally, once the scan is running, the application will accept two commands from the console: dump and exit.

- Dump instructs the application to dump the filenames in the container to the console and then clear it.
- Exit instructs the application to free resources then exit immediately.

Requirements

- Your application must be written in either C or C++.
- Each substring will be serviced by a dedicated thread.
- The periodic dumper will be a dedicated thread.
- A single container will be shared by all threads.
- You must ensure that your application releases all resources you allocated before you terminate - i. e. don’t rely on program termination to clean things up.
- You may use no libraries other than what is provided by the OS and your toolchain’s CRT & STL.

Assumptions

In the interest of keeping the scope of this problem manageable while still allowing you to demonstrate your knowledge of important topics like synchronization, you should make the following assumptions in designing your solution.

- File system contents are static during a run.
- You have the required access to all files and directories.
- Path lengths are limited to 255 characters.
- Paths are encoded in your OS's native string representation.
- You should not specially handle symlinks or reparse points and there are no circular paths.
- Substrings specified on the command line do not overlap.

Evaluation Criteria

Your solution will be evaluated on the following criteria in descending order of importance.

- Correctness - Does it work correctly?
- Simplicity - Does your solution solve the problem without making things more complicated than necessary?
- Robustness - Does your program leak any resources?
- Synchronization - Does it make proper use of synchronization primitives appropriate for the problem?
- Error Handling - Does your solution handle and report the major errors?
- Comments and Readability - Is your code clear and concise?

Miscellaneous Notes

- We encourage you to make simplifying assumptions rather than attempting to handle every edge case. If you do so, please document them so we can see your reasoning.
- While we value tests, in order to save your time, we do not encourage you to submit tests. We *DO* encourage you to describe how you'd test and what your test cases would be.
- Something to think about for the interviews: what would you do if you had more time for this exercise?